Flutter Practicals.

Laboratory record · Mobile Application Development · Flutter & Dart

A working record of the ten yellow-marked experiments from the laboratory syllabus. Each experiment carries its aim, the concepts at play, one complete and lean main.dart, and the output you should see when it runs. Paste, run, tick.

CONTENTS - TEN EXPERIMENTS

01 Hello World & the Toolchain
02 Parent-to-Child State
03 Login Form Validation
04 Popup Menus & Drawer
05 Dynamic Dropdown
06 GridView Layout
07 ListView, Custom Widget
08 Expanded & Flexible
09 Video Playback Plugin
10 Gestures & Callbacks

Getting the Toolchain Ready

INSTALLING THE SDK

Flutter ships as a single SDK folder plus a doctor that audits the installation. Download the stable SDK archive, extract it to a permanent location such as ~/development/flutter, and add its bin directory to PATH in your shell profile. flutter doctor then reports exactly what is missing; on this lab's setup that is Android Studio, which bundles the Android SDK, platform tools and the emulator, plus the licences Android asks you to accept once.

```
$ export PATH="$PATH:$HOME/development/flutter/bin"
$ flutter doctor
$ flutter doctor --android-licenses
```

ONE DEVICE, ONE PROJECT

Create one virtual device in Android Studio: open Device Manager, choose Create Virtual Device, and pick a Pixel 7 image with a recent API level. Launch it once and leave it running; flutter devices should list it. A physical phone works just as well after enabling Developer options and USB debugging, with no emulator required.

```
$ flutter create hello_world
$ cd hello_world
$ flutter run
```

while running: r hot reload · R hot restart · q quit

WHAT MATTERS IN A SCAFFOLDED PROJECT

Every experiment in this record is one complete main.dart. Paste it into lib/main.dart of a fresh flutter create project, run flutter pub get whenever a note lists a dependency, and hot reload. Four locations in the scaffolded project matter:

LOCATION	ROLE
lib/main.dart	App entry point. main() calls runApp() with the root widget of the experiment.
pubspec.yaml	Dependencies, assets and fonts. Edit it, then run flutter pub get.
android/ · ios/	Native platform shells. Permissions, such as INTERNET in Experiment 09, live in their manifests.
test/	Widget tests. Update or delete the scaffolded counter test before running flutter test.

COMMAND VOCABULARY

The commands below cover everything this record uses. Project names must be snake_case, because they become Dart package names and the toolchain refuses anything else.

COMMAND	MEANING
`flutter doctor`	Audits the toolchain and lists the missing pieces.
`flutter create <name>`	Scaffolds a new project; snake_case names only.
`flutter run`	Builds and installs the app on the connected device.
`flutter pub add <pkg>`	Adds a dependency to pubspec.yaml and fetches it.
`flutter analyze`	Static analysis; catches mistakes before the build does.

EXPERIMENT 01 SYLLABUS ITEM 1

Hello World and the Toolchain

AIM

To develop a minimal Flutter application that renders a single screen, and to walk the complete toolchain path this course depends on: project creation, Flutter SDK configuration, widget composition, emulator launch and running the app.

CONCEPTS AT PLAY

Every Flutter app starts at main(), the first Dart function the engine calls. runApp() attaches a widget to the screen by making it the root of the widget tree. MaterialApp supplies app-level plumbing (navigation, theming, the debug banner) and Scaffold gives a page its physical layout: app bar, body, floating button. Text, Center and AppBar are widgets too, because in Flutter the entire interface is one tree of composable widgets with no separate markup language. The SDK is configured once and audited with flutter doctor; from then on the emulator compiles and installs the app, and hot reload pushes code changes into the running process in under a second.

runApp · MaterialApp · Scaffold · widget tree · hot reload

CODE - LIB/MAIN.DART

LIB/MAIN.DART DART

```dart
import 'package:flutter/material.dart';

void main() => runApp(const HelloApp());

// The root widget: immutable, created once per launch.
class HelloApp extends StatelessWidget {
  const HelloApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Hello World',
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        appBar: AppBar(title: const Text('Experiment 01')),
        body: const Center(
          child: Text(
            'Hello, Flutter!',
```

```
            style: TextStyle(fontSize: 28, fontWeight: FontWeight.w600),
          ),
        ),
      ),
    );
  }
}
```

EXPECTED OUTPUT

> A blue app bar reading 'Experiment 01' over a white screen with 'Hello, Flutter!' centred in 28 px type. No interaction yet: the app renders one static frame.

RESULT Project created with flutter create, compiled, installed and launched on the Android emulator.

EXPERIMENT 02 SYLLABUS ITEM 6

Parent-to-Child State Management

AIM

To demonstrate how state is owned and moved between a parent StatefulWidget and its stateless children: state travels down as constructor data, user events travel back up through callbacks.

CONCEPTS AT PLAY

Flutter uses one-way data flow. The parent _CounterAppState owns the integer; children never mutate it themselves. CounterDisplay receives the count as a plain final field and re-renders whenever the parent rebuilds. CounterButton receives a VoidCallback instead, so it reports the tap upward and lets the parent decide what changing the state means. Calling setState() marks that State object dirty and schedules a rebuild of exactly its subtree. This discipline, data down and events up, is the foundation that forms, dropdowns and lists quietly rely on in the experiments that follow.

`StatefulWidget · setState · callback · rebuild cycle`

CODE - LIB/MAIN.DART

LIB/MAIN.DART DART

```dart
import 'package:flutter/material.dart';

void main() => runApp(const CounterApp());

class CounterApp extends StatefulWidget {
  const CounterApp({super.key});

  @override
  State<CounterApp> createState() => _CounterAppState();
}

// PARENT - owns the state and decides what it means.
class _CounterAppState extends State<CounterApp> {
  int _count = 0;

  void _increment() => setState(() => _count++);
```

```
  @overrideWidget build(BuildContext context) {
    returnMaterialApp(
      home: Scaffold(
        appBar: AppBar(title: constText('Parent-Child State')),
        body: Center(
          // Data flows DOWN: the child only displays what it receives.
          child: CounterDisplay(count: _count),
        ),
        // Events flow UP: the child reports taps through a callback.
        floatingActionButton: CounterButton(onPressed: _increment),
      ),
    );
  }
}

// CHILD 1 - stateless; renders the data it was
given.classCounterDisplayextendsStatelessWidget {
  constCounterDisplay({super.key, requiredthis.count});

  finalint count;

  @overrideWidget build(BuildContext context) {
    returnText('$count',
        style: constTextStyle(fontSize: 56, fontWeight: FontWeight.w600));
  }
}

// CHILD 2 - stateless; owns no data, only reports
events.classCounterButtonextendsStatelessWidget {
  constCounterButton({super.key, requiredthis.onPressed});

  finalVoidCallback onPressed;

  @overrideWidget build(BuildContext context) {
    returnFloatingActionButton(onPressed: onPressed,
        child: constIcon(Icons.add));
  }
}
```

EXPECTED OUTPUT

> An app bar, a large centred counter starting at 0 and a floating action button. Every press increments the number by one; the button itself never holds any data.

RESULT Counter state lifted to the parent; increments verified with hot reload on the emulator.

EXPERIMENT 03 SYLLABUS ITEM 9

Login Form with Validation

AIM

CONCEPTS AT PLAY

A Form groups fields so they validate together. Its GlobalKey lets the submit handler reach into the form state and call validate(), which runs every child TextFormField's validator and shows inline error text beneath the failing fields. A validator returns null when the value passes and a message string when it fails; that string is exactly what the user sees. obscureText masks the password, keyboardType selects the mobile keyboard, and TextEditingController reads the committed values at submit time. dispose() releases both controllers when the screen is destroyed, the standard cleanup for any stateful resource.

`Form` · `GlobalKey` · `validator` · `TextEditingController`

CODE - LIB/MAIN.DART

LIB/MAIN.DART DART

```dart
import 'package:flutter/material.dart';

void main() => runApp(const LoginApp());

class LoginApp extends StatelessWidget {
  const LoginApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: const Text('Sign In')),
        body: const LoginForm(),
      ),
    );
  }
}

class LoginForm extends StatefulWidget {
  const LoginForm({super.key});

  @override
  State<LoginForm> createState() => _LoginFormState();
}

class _LoginFormState extends State<LoginForm> {
  final _formKey = GlobalKey<FormState>();
  final _email = TextEditingController();
  final _password = TextEditingController();

  @override
  void dispose() {
    _email.dispose();
    _password.dispose();
    super.dispose();
  }

  void _submit() {
    // validate() runs every validator; true only when all of them pass.
    if (_formKey.currentState!.validate()) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Welcome, ${_email.text}')),
      );
```

```
    }
  }

  @override Widget build(BuildContext context) {
    return SingleChildScrollView(
      padding: const EdgeInsets.all(24),
      child: Form(
        key: _formKey,
        child: Column(
          children: [
            TextFormField(
              controller: _email,
              keyboardType: TextInputType.emailAddress,
              decoration: const InputDecoration(
                  labelText: 'Email', border: OutlineInputBorder()),
              validator: (v) => (v == null || !v.contains('@'))
                  ? 'Enter a valid email' : null,
            ),
            const SizedBox(height: 16),
            TextFormField(
              controller: _password,
              obscureText: true,
              decoration: const InputDecoration(
                  labelText: 'Password', border: OutlineInputBorder()),
              validator: (v) => (v == null || v.length < 6)
                  ? 'Minimum 6 characters' : null,
            ),
            const SizedBox(height: 24),
            FilledButton(onPressed: _submit, child: const Text('SIGN IN')),
          ],
        ),
      ),
    );
  }
}
```

EXPECTED OUTPUT

> Two outlined fields with floating labels. Submitting while empty shows 'Enter a valid email' and 'Minimum 6 characters' beneath the fields; a valid pair raises a SnackBar greeting the address.

RESULT Both validators fire on empty input; valid credentials produce the SnackBar acknowledgement.

EXPERIMENT 04 SYLLABUS ITEM 11

App Menus: Popup and Drawer

AIM

To create and use two menu mechanisms, an overflow PopupMenuButton in the app bar and a navigation Drawer, and to route their selections back into the screen state.

CONCEPTS AT PLAY

and is dismissed with Navigator.pop(context) before the app applies the tapped destination. Menus are therefore just widgets: their items are ordinary ListTiles or PopupMenuItems and their behaviour is ordinary callbacks. Every selection is written into the app bar title so both mechanisms show visible state.

`PopupMenuButton` · `Drawer` · `Navigator.pop` · `onSelected`

CODE - LIB/MAIN.DART

LIB/MAIN.DART DART

```dart
import 'package:flutter/material.dart';

void main() => runApp(const MenuApp());

class MenuApp extends StatelessWidget {
  const MenuApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(home: const MenuPage());
  }
}

class MenuPage extends StatefulWidget {
  const MenuPage({super.key});

  @override
  State<MenuPage> createState() => _MenuPageState();
}

class _MenuPageState extends State<MenuPage> {
  String _screen = 'Home';
  String _sort = 'Rating';
  final Map<String, IconData> _nav = {
    'Home': Icons.home,
    'Profile': Icons.person,
    'Settings': Icons.settings,
  };

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('$_screen  -  sorted by $_sort'),
        actions: [
          PopupMenuButton<String>(
            tooltip: 'Sort',
            onSelected: (value) => setState(() => _sort = value),
            itemBuilder: (_) => const [
              PopupMenuItem(value: 'Rating', child: Text('Sort by rating')),
              PopupMenuItem(value: 'Price', child: Text('Sort by price')),
              PopupMenuItem(value: 'Newest', child: Text('Sort by newest')),
            ],
          ),
        ],
      ),
      drawer: Drawer(
        child: ListView(
          children: [
```

```
            constDrawerHeader(
              decoration: BoxDecoration(color: Color(0xFF216B91)),
              child: Text('Flutter Menus',
                  style: TextStyle(color: Colors.white, fontSize: 22)),
            ),
            for (final entry in _nav.entries)
              ListTile(
                leading: Icon(entry.value),
                title: Text(entry.key),
                onTap: () {
                  Navigator.pop(context); // close the drawer first
                  setState(() ⇒ _screen = entry.key);
                },
              ),
          ],
        ),
      ),
      body: constCenter(
          child: Text('Both menus write their choice into the app bar')),
    );
  }
}
```

EXPECTED OUTPUT

> A sort icon in the app bar opens a three-item popup; picking one updates the title line. The hamburger opens a drawer with a coloured header and three destinations; tapping one closes the drawer and retitles the app bar.

RESULT Popup selection and drawer navigation both persist through setState and survive hot reload.

EXPERIMENT 05 SYLLABUS ITEM 13

Dropdown Filled from Dynamic Data

AIM

To populate a DropdownButton from a dynamic List instead of hard-coded children, and to react to selection changes through setState.

CONCEPTS AT PLAY

The dropdown is data-driven: every entry of _cities is mapped to a DropdownMenuItem, so adding a city to the list is enough to change the menu with no widget edits. value must match one of the item values, or stay null so the hint shows, and onChanged fires with the newly chosen string. Because the screen is stateful, setState stores the choice and triggers the rebuild that updates the caption below. This map-data-to-items pattern is the same one ListView.builder and GridView.builder use later, which is why it is worth learning on a small widget first.

DropdownButton · DropdownMenuItem · onChanged · hint

CODE - LIB/MAIN.DART

LIB/MAIN.DART DART

```
import 'package:flutter/material.dart';
```

```dart
void main() => runApp(const CityApp());

class CityApp extends StatelessWidget {
  const CityApp({super.key});

  @override Widget build(BuildContext context) {
    return MaterialApp(home: const CityPicker());
  }
}

class CityPicker extends StatefulWidget {
  const CityPicker({super.key});

  @override State<CityPicker> createState() => _CityPickerState();
}

class _CityPickerState extends State<CityPicker> {
  // The data source: edit this list and the menu follows.final List<String> _cities =
['Mumbai', 'Delhi', 'Bengaluru', 'Chennai'];
  String? _selected;

  @override Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Dynamic Dropdown')),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            DropdownButton<String>(
              hint: const Text('Select a city'),
              value: _selected,
              items: _cities
                  .map((city) => DropdownMenuItem<String>(
                      value: city, child: Text(city)))
                  .toList(),
              onChanged: (value) => setState(() => _selected = value),
            ),
            const SizedBox(height: 24),
            Text(_selected == null
                ? 'Nothing selected yet'
                : 'You picked $_selected'),
          ],
        ),
      ),
    );
  }
}
```

EXPECTED OUTPUT

> A centred dropdown showing 'Select a city'. Choosing an entry closes the menu, replaces the hint with the city name and updates the caption beneath to 'You picked '.

RESULT Menu renders from the list; selection state verified by the live caption under the widget.

GridView Layout

AIM

To display a collection of items in a two-column grid using GridView.builder, with every cell built on demand from an in-memory data list.

CONCEPTS AT PLAY

GridView.builder is lazy: it constructs only the cells visible on screen and recycles them while scrolling, which is why it scales to thousands of items. SliverGridDelegateWithFixedCrossAxisCount fixes the geometry, two columns with 12 px spacing on both axes, and the delegate computes each cell size automatically. itemBuilder receives the index and reads its data, an icon plus a label, from the list, which keeps data and layout code strictly separated. Card supplies the outline, and a centred Column stacks the icon and caption inside each cell.

GridView.builder · SliverGridDelegate · itemBuilder · lazy building

CODE - LIB/MAIN.DART

LIB/MAIN.DART DART

```dart
import 'package:flutter/material.dart';

void main() => runApp(const GridApp());

class GridApp extends StatelessWidget {
  const GridApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(home: const ToolGrid());
  }
}

class ToolGrid extends StatelessWidget {
  const ToolGrid({super.key});

  static const List<Map<String, dynamic>> _tools = [
    {'icon': Icons.camera_alt, 'label': 'Camera'},
    {'icon': Icons.music_note, 'label': 'Music'},
    {'icon': Icons.map, 'label': 'Maps'},
    {'icon': Icons.mail, 'label': 'Mail'},
    {'icon': Icons.cloud, 'label': 'Weather'},
    {'icon': Icons.access_time, 'label': 'Clock'},
    {'icon': Icons.fitness_center, 'label': 'Fitness'},
    {'icon': Icons.shopping_cart, 'label': 'Shop'},
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('GridView.builder')),
      body: GridView.builder(
        padding: const EdgeInsets.all(12),
        gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
          crossAxisCount: 2,
```

```
            crossAxisSpacing: 12,
            mainAxisSpacing: 12,
          ),
          itemCount: _tools.length,
          itemBuilder: (context, index) {
            final tool = _tools[index];
            returnCard(
              elevation: 0,
              shape: RoundedRectangleBorder(
                borderRadius: BorderRadius.circular(8),
                side: constBorderSide(color: Color(0xFFB0C3CD)),
              ),
              child: Column(
                mainAxisAlignment: MainAxisAlignment.center,
                children: [
                  Icon(tool['icon'], size: 40, color: constColor(0xFF216B91)),
                  constSizedBox(height: 10),
                  Text(tool['label'] asString,
                      style: constTextStyle(fontWeight: FontWeight.w500)),
                ],
              ),
            );
          },
        ),
      );
    }
  }
```

EXPECTED OUTPUT

> A two-column grid of eight outlined square tiles, each showing a blue icon over its label. The grid scrolls whenever the screen is too short for all tiles.

RESULT Eight cells built lazily from the data list; grid reflows correctly on rotation.

EXPERIMENT 07 SYLLABUS ITEM 17

ListView with a Custom Widget

AIM

To render dynamic data through a hand-made reusable widget: a typed Contact model, one custom ContactTile widget, and ListView wiring them together.

CONCEPTS AT PLAY

The data lives in a small model class, Contact, so the list is typed rather than a pile of maps. ContactTile is a custom StatelessWidget with a required final field: the same data-down rule as Experiment 02, now applied to a list row. ListView.separated asks for one tile per index and the tile alone decides how to draw initials, name and phone. Because the row is a real widget, changing its design later means editing one class instead of five copy-pasted rows. This model plus custom widget plus builder trio is the standard shape of every list-driven screen in Flutter.

LIB/MAIN.DART DART

```dart
import 'package:flutter/material.dart';

void main() => runApp(const ContactApp());

// The data model - a typed record of one contact.
class Contact {
  const Contact(this.name, this.phone, this.color);

  final String name;
  final String phone;
  final Color color;

  String get initials => name
      .split(' ')
      .map((part) => part[0])
      .take(2)
      .join();
}

const List<Contact> _contacts = [
  Contact('Aisha Khan', '+91 98200 11223', Color(0xFF216B91)),
  Contact('Rohan Mehta', '+91 98111 44556', Color(0xFF377F4F)),
  Contact('Sara Iyer', '+91 99001 78901', Color(0xFF947941)),
  Contact('Dev Patel', '+91 98234 55670', Color(0xFF4E6A78)),
  Contact('Meera Nair', '+91 97654 32109', Color(0xFF904942)),
];

// The CUSTOM component: one reusable row drawn from its fields.
class ContactTile extends StatelessWidget {
  const ContactTile({super.key, required this.contact});

  final Contact contact;

  @override
  Widget build(BuildContext context) {
    return ListTile(
      leading: CircleAvatar(
        backgroundColor: contact.color,
        child: Text(contact.initials,
            style: const TextStyle(color: Colors.white)),
      ),
      title: Text(contact.name,
          style: const TextStyle(fontWeight: FontWeight.w600)),
      subtitle: Text(contact.phone),
    );
  }
}

class ContactApp extends StatelessWidget {
  const ContactApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
```

```
        appBar: AppBar(title: constText('Contacts')),
        body: ListView.separated(
          itemCount: _contacts.length,
          separatorBuilder: (_, __) ⇒ constDivider(height: 1),
          itemBuilder: (context, index) ⇒
              ContactTile(contact: _contacts[index]),
        ),
      ),
    );
  }
}
```

EXPECTED OUTPUT

> A scrolling list of five contact rows, each with a coloured circular avatar holding the person's initials, a bold name and a muted phone number, separated by hairline dividers.

RESULT Custom tile reused for all five rows; one class change restyles the entire list.

EXPERIMENT 08 SYLLABUS ITEM 19

Expanded and Flexible Responsive Layout

AIM

To design responsive layouts by controlling how Row and Column distribute their leftover space with Expanded, Flexible and flex factors.

CONCEPTS AT PLAY

Row and Column give children only their intrinsic size by default; anything unconstrained overflows with yellow-black stripes. Expanded forces a child to fill its share of the leftover space: flex 2 claims twice the remainder of flex 1, as the first row shows. Flexible with FlexFit.loose is politer, it lets a child be at most its share but no larger, which is what the middle row contrasts. The final row wraps a long paragraph inside Expanded so the text receives a bounded width and wraps instead of pushing siblings away. Fixed-width siblings are measured first; the flex children split whatever remains.

flex factor · FlexFit.loose · remaining space · constraint

CODE - LIB/MAIN.DART

LIB/MAIN.DART DART

```dart
import 'package:flutter/material.dart';

void main() ⇒ runApp(const FlexApp());

class FlexApp extends StatelessWidget {
  const FlexApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: const Text('Expanded & Flexible')),
        body: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
```

```
          children: [
            // 1. Fixed box, then flex children splitting the remainder 1:2.Row(
              children: [
                Container(width: 90, height: 84,
                    color: constColor(0xFF9AA0A4)),
                Expanded(
                  child: Container(
                    height: 84, color: constColor(0xFF216B91),
                    alignment: Alignment.center,
                    child: constText('flex 1',
                        style: TextStyle(color: Colors.white)),
                  ),
                ),
                Expanded(
                  flex: 2,
                  child: Container(
                    height: 84, color: constColor(0xFF4E6A78),
                    alignment: Alignment.center,
                    child: constText('flex 2',
                        style: TextStyle(color: Colors.white)),
                  ),
                ),
              ],
            ),
            constSizedBox(height: 14),
            // 2. Flexible(loose) caps a child; Expanded forces it to fill.Row(
              children: [
                Flexible(
                  fit: FlexFit.loose,
                  child: Container(height: 84, width: 180,
                      color: constColor(0xFF377F4F)),
                ),
                Expanded(
                  child: Container(height: 84,
                      color: constColor(0xFF216B91)),
                ),
              ],
            ),
            constSizedBox(height: 14),
            // 3. Long text is bounded by Expanded, so it wraps, never overflows.Row(
              children: [
                Expanded(
                  child: Container(
                    padding: constEdgeInsets.all(12),
                    color: constColor(0xFFEEF2F4),
                    child: constText(
                      'Expanded bounds this paragraph, so it wraps inside the ''remaining
width instead of pushing its siblings away.',
                    ),
                  ),
                ),
              ],
            ),
          ],
        ),
      ),
    );
```

```
  }
}
```

EXPECTED OUTPUT

> Three stacked rows: a fixed 90 px grey block beside blue blocks sized 1:2; a green block capped at 180 px beside a fully filling blue block; and a paragraph wrapped neatly inside the final row's width.

RESULT Flex ratios verified: flex 2 occupies exactly twice the remainder of flex 1 at any screen width.

EXPERIMENT 09 SYLLABUS ITEM 20

Media Playback with a Plugin

AIM

To play a remote video with the video_player plugin: loading, initialising, and controlling playback with a custom play/pause button and a scrubber.

CONCEPTS AT PLAY

Media playback is not part of the Flutter core; it arrives as a platform plugin whose Dart API talks to native code over message channels. VideoPlayerController loads a URL, initialises the decoder on the platform side, and reports state through a ChangeNotifier: value.isInitialized, value.isPlaying and value.aspectRatio. The UI reads those flags and rebuilds with setState; initState creates the controller and dispose() releases it, forgetting that call is the classic memory leak. Audio-only playback follows the same lifecycle with the audioplayers package, whose AudioPlayer().play() accepts a URL or a bundled asset source.

platform plugin · controller lifecycle · AspectRatio · ValueNotifier

CODE - LIB/MAIN.DART

LIB/MAIN.DART DART

```dart
import 'package:flutter/material.dart';
import 'package:video_player/video_player.dart';

void main() => runApp(const MediaApp());

class MediaApp extends StatelessWidget {
  const MediaApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(home: const PlayerScreen());
  }
}

class PlayerScreen extends StatefulWidget {
  const PlayerScreen({super.key});

  @override
  State<PlayerScreen> createState() => _PlayerScreenState();
}

class _PlayerScreenState extends State<PlayerScreen> {
  late final VideoPlayerController _controller;
```

```
  @overridevoid initState() {
    super.initState();
    _controller = VideoPlayerController.networkUrl(Uri.parse(
      'https://storage.googleapis.com/gtv-videos-bucket/sample/ForBiggerBlazes.mp4',
    ))..initialize().then((_) => setState(() {}));
  }

  @overridevoid dispose() {
    _controller.dispose(); // release the decoder - never skip thissuper.dispose();
  }

  @overrideWidget build(BuildContext context) {
    returnScaffold(
      appBar: AppBar(title: constText('Video Player')),
      body: Column(
        children: [
          _controller.value.isInitialized
              ? AspectRatio(
                  aspectRatio: _controller.value.aspectRatio,
                  child: VideoPlayer(_controller),
                )
              : constPadding(
                  padding: EdgeInsets.all(32),
                  child: Center(child: CircularProgressIndicator()),
                ),
          VideoProgressIndicator(_controller, allowScrubbing: true),
          IconButton(
            iconSize: 40,
            icon: Icon(_controller.value.isPlaying
                ? Icons.pause : Icons.play_arrow),
            onPressed: () => setState(() {
              _controller.value.isPlaying
                  ? _controller.pause()
                  : _controller.play();
            }),
          ),
        ],
      ),
    );
  }
}
```

NOTE Before running: flutter pub add video_player, then add the internet permission to android/app/src/main/AndroidManifest.xml inside the manifest tag: <uses-permission android:name="android.permission.INTERNET"/>

EXPECTED OUTPUT

> A spinner while the clip buffers, then the video at its true aspect ratio with a draggable progress bar. The play icon flips to pause and back as playback state changes.

RESULT Remote MP4 decoded through the plugin; play, pause and scrub all drive the progress indicator.

Gestures and Button Callbacks

AIM

To capture raw gestures on an arbitrary widget with GestureDetector, covering tap, double-tap and long-press, and to compare them with the declarative onPressed callbacks of buttons.

CONCEPTS AT PLAY

Buttons own their gesture handling; GestureDetector hands the same power to any widget. Each callback sits on a distinct recogniser: onTap fires on a quick press-and-release, onDoubleTap on two taps inside the platform threshold, and onLongPress after a sustained hold. Because several recognisers compete for the same pointer, the widget holds a gesture arena and the winning recogniser claims the event, which is exactly why tap and double-tap can coexist on one card. The filled button shows the declarative alternative: a single onPressed callback with no recognisers to manage. A log line records every event so the timing of each recogniser is observable during the demo.

gesture arena · recogniser · onTap / onDoubleTap / onLongPress

CODE - LIB/MAIN.DART

LIB/MAIN.DART DART

```dart
import 'package:flutter/material.dart';

void main() => runApp(const GestureApp());

class GestureApp extends StatelessWidget {
  const GestureApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(home: const GestureLab());
  }
}

class GestureLab extends StatefulWidget {
  const GestureLab({super.key});

  @override
  State<GestureLab> createState() => _GestureLabState();
}

class _GestureLabState extends State<GestureLab> {
  String _log = 'Awaiting input...';

  void _record(String event) => setState(() => _log = event);

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('GestureDetector')),
      body: Padding(
        padding: const EdgeInsets.all(20),
        child: Column(
```

```
            children: [
              GestureDetector(
                onTap: () => _record('onTap fired'),
                onDoubleTap: () => _record('onDoubleTap fired'),
                onLongPress: () => _record('onLongPress fired'),
                child: Container(
                  height: 150,
                  alignment: Alignment.center,
                  decoration: BoxDecoration(
                    color: const Color(0xFFEEF2F4),
                    borderRadius: BorderRadius.circular(10),
                    border: Border.all(color: const Color(0xFFB0C3CD)),
                  ),
                  child: const Text('Tap - double-tap - long-press me',
                      style: TextStyle(fontWeight: FontWeight.w500)),
                ),
              ),
              const SizedBox(height: 20),
              FilledButton(
                onPressed: () => _record('Button onPressed fired'),
                child: const Text('Ordinary button'),
              ),
              const SizedBox(height: 24),
              Text(_log, style: const TextStyle(fontStyle: FontStyle.italic)),
            ],
          ),
        ),
      );
  }
}
```

EXPECTED OUTPUT

> A tappable card and a button above a live log line. A quick tap, a double-tap and a held press each print their own event; pressing the button reports that its onPressed fired.

RESULT All three recognisers distinguished on one widget; button callback recorded through the same log.